RAPPORT DE PROJET

CENTRALE DCC SUR FPGA

Système MicroBlaze & Interface AXI

Réalisé par :
Louis DAI 21204918
Asmar MOHAMED 21208357

Année Universitaire 2025-2026

Partie Hardware

Explication des fichiers et des entrées et sorties des architectures :

1. Generateur_Trames :
Fonction : Permet de créer des trames de taille 51 en fonction de l'interrupteur déclenché.

Signature :

```
entity DCC_FRAME_GENERATOR is
    Port ( Interrupteur : in STD_LOGIC_VECTOR(7 downto 0);
           Trame_DCC    : out STD_LOGIC_VECTOR(50 downto 0));
end DCC_FRAME_GENERATOR;
```

Explication des signaux :

- `Interrupteur` : Les entrées, interrupteurs de la carte
- `Trame_DCC` : Trame de sortie en fonction de l'interrupteur actionné

2. Registre_DCC :
Fonction : Charger ou décaler la trame pour l'envoyer à la MAE.

Signature :

```
entity REGISTRE_DCC is
    Port ( clk, reset : in STD_LOGIC;
           Trame_DCC : in STD_LOGIC_VECTOR (50 downto 0);
           COM_REG   : in STD_LOGIC_VECTOR (1 downto 0);
           Reg_DCC   : out STD_LOGIC
          );
end REGISTRE_DCC;
```

Explication des signaux :

- `clk` : Horloge 100 MHz (celle de la carte)
- `reset` : Signal de réinitialisation
- `Trame_DCC` : Trame d'entrée, qui va être chargée
- `COM_REG` : Signal envoyé par la MAE pour soit charger, soit décaler la trame (*load* : 10, *shift* : 01)
- `Reg_DCC` : Sortie du bit de poids fort de la trame décalée

3. DCC_BIT0 :
Fonction : Envoyer un bit 0 du protocole DCC.

Signature :

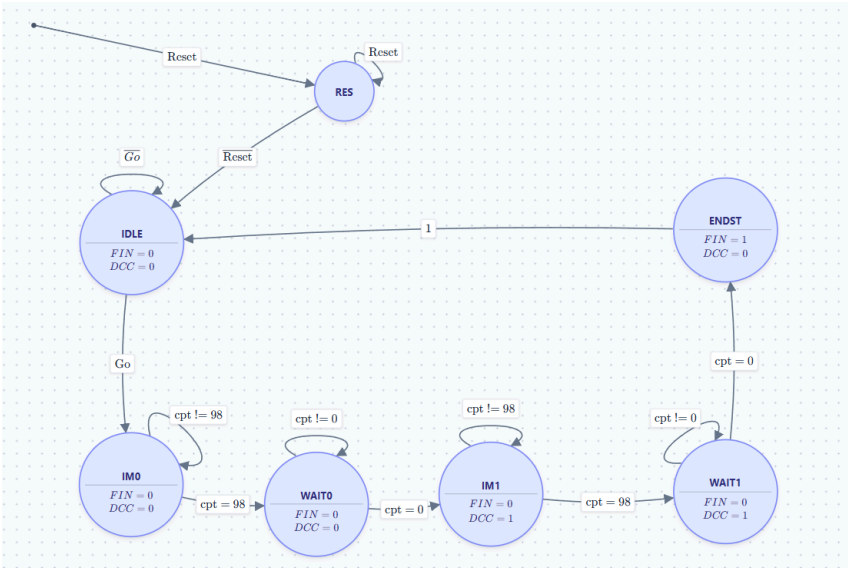
```
entity DCC_BIT0 is
    Port ( Clk1M : in STD_LOGIC;
           Clk : in STD_LOGIC;
           Reset : in STD_LOGIC;
           Go : in STD_LOGIC;
           Fin : out STD_LOGIC;
```

```
            DCC : out STD_LOGIC
            );
end DCC_BIT0;
```

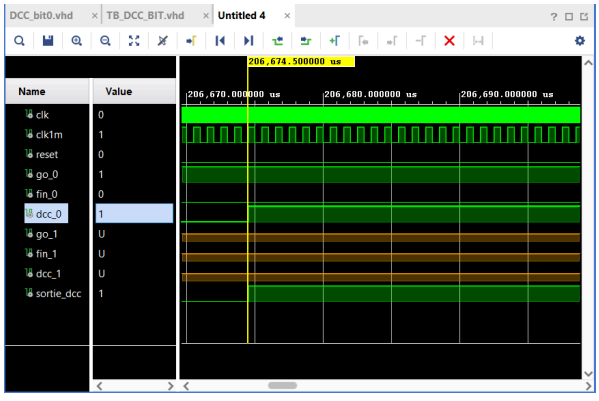
Explication des signaux :

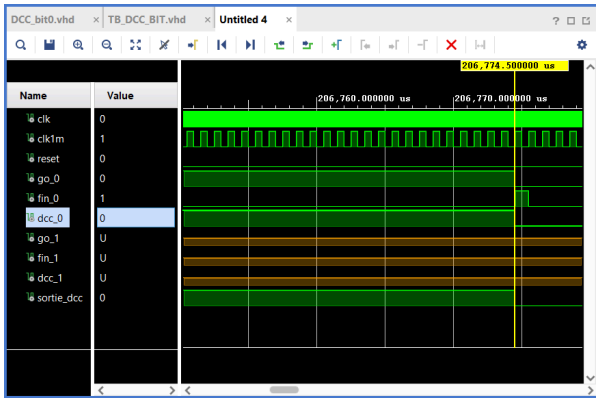
- `Clk1M` : L'horloge 1 MHz, pour le compteur
- `Clk` : Horloge 100 MHz, pour tous les modules
- `Reset` : Signal de réinitialisation
- `Go` : Signal d'activation du module (1 : activé, 0 : IDLE)
- `Fin` : Signal pour indiquer la fin (1 : fin, 0 : non fini)
- `DCC` : Bit de sortie du 0 selon le protocole DCC (100 µs à 0 puis 100 µs à 1)

MAE :

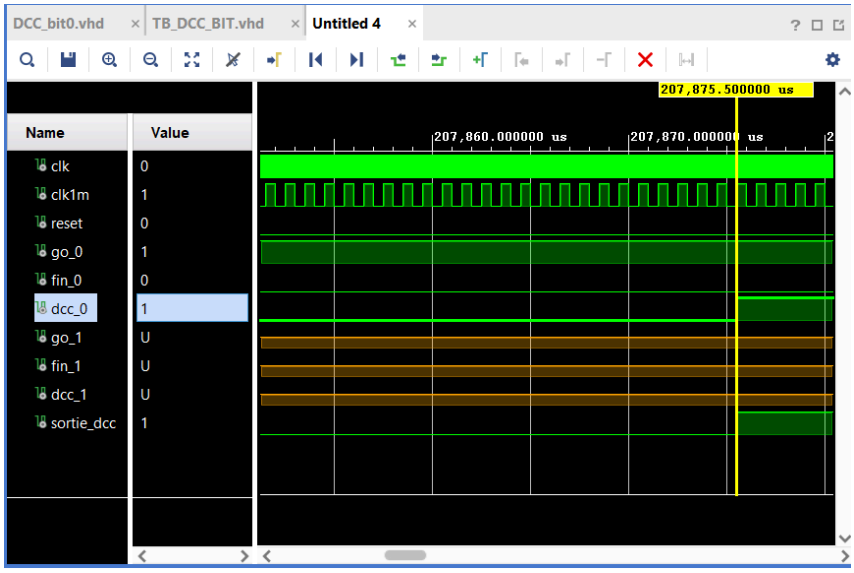


TestBench :





Il y a un délai de 1 ms entre chaque envoi de bit. L'attente préalable de 1 ms avant chaque envoi introduit un décalage global de 1 ms, indépendamment de la durée de transmission du bit lui-même.



On remarque bien que l'envoi du bit 0 sur le protocole DCC fonctionne.

4. DCC_BIT1 :

Fonction : Envoyer un bit 1 du protocole DCC.

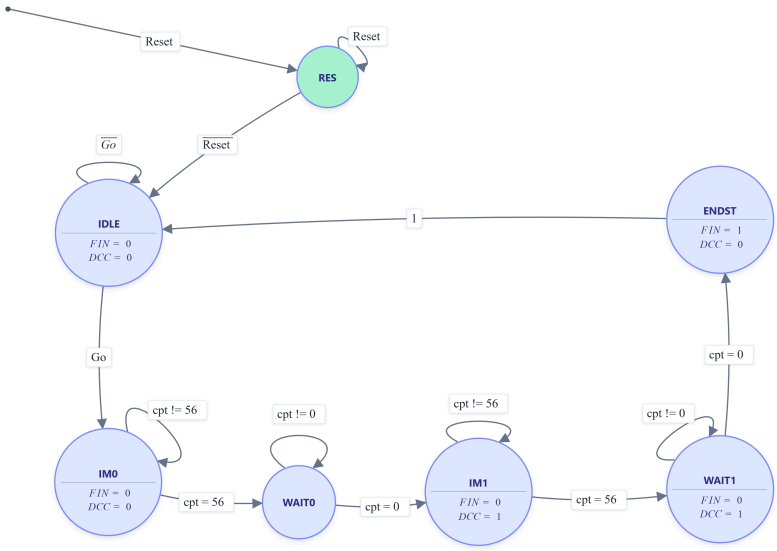
Signature :

```
entity DCC_BIT1 is
   Port ( Clk1M : in STD_LOGIC;
          Clk : in STD_LOGIC;
          Reset : in STD_LOGIC;
          Go : in STD_LOGIC;
          Fin : out STD_LOGIC;
          DCC : out STD_LOGIC);
end DCC_BIT1;
```

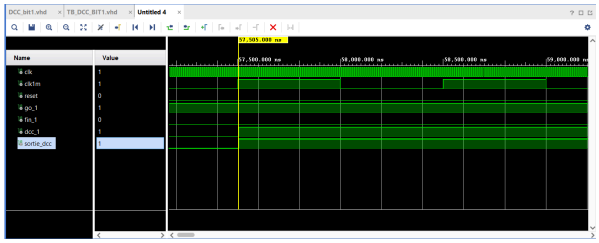
Explication des signaux :

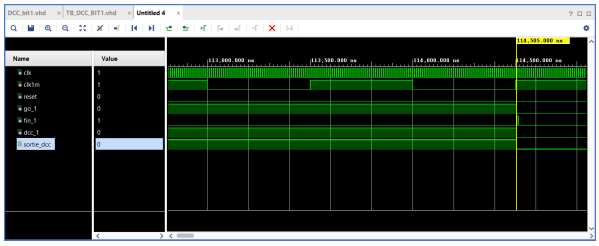
- `Clk1M` : L'horloge 1 MHz, pour le compteur
- `Clk` : Horloge 100 MHz, pour tous les modules
- `Reset` : Signal de réinitialisation
- `Go` : Signal d'activation du module (1 : activé, 0 : IDLE)
- `Fin` : Signal pour indiquer la fin (1 : fin, 0 : pas fini)
- `DCC` : Bit de sortie du 1 selon le protocole DCC (58 µs à 0 puis 58 µs à 1)

MAE :

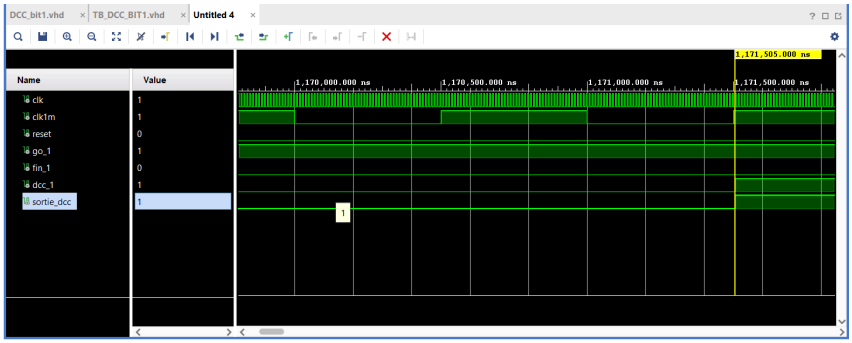


TestBench :





De même ici, il y a un délai de 1 ms entre chaque envoi de bit.



On remarque bien que l'envoi du bit 1 sur le protocole *DCC* fonctionne.

5. MAE :

Fonction : Orchestre la séquence de transmission des trames de données à travers une succession d'états logiques définis.

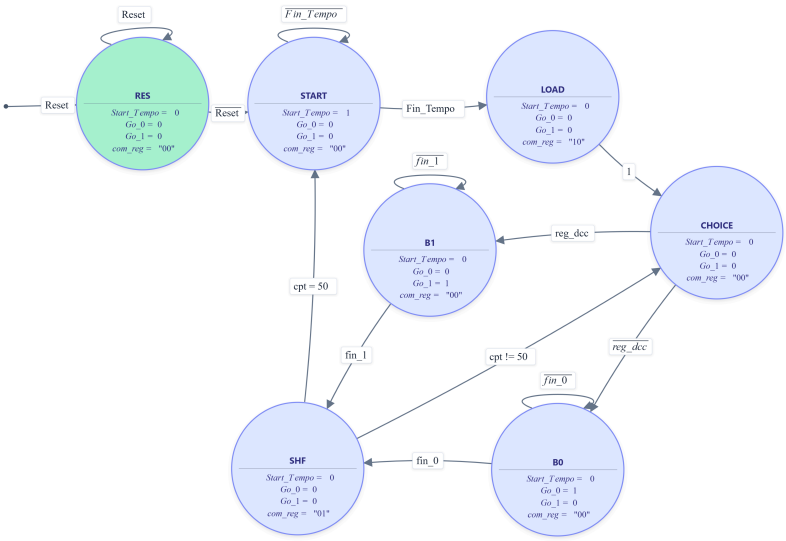
Signature :

```
entity MAE is
    Port ( Clk         : in STD_LOGIC;
           Reset       : in STD_LOGIC;
           Fin_Tempo   : in STD_LOGIC;
           Fin_0       : in STD_LOGIC;
           Fin_1       : in STD_LOGIC;
           Reg_DCC     : in STD_LOGIC;
           Start_Tempo : out STD_LOGIC;
           Go_0        : out STD_LOGIC;
           Go_1        : out STD_LOGIC;
           Com_Reg     : out STD_LOGIC_VECTOR (1 downto 0));
           -- bit 1 : load, bit 0 : shift
end MAE;
```

Explication des signaux :

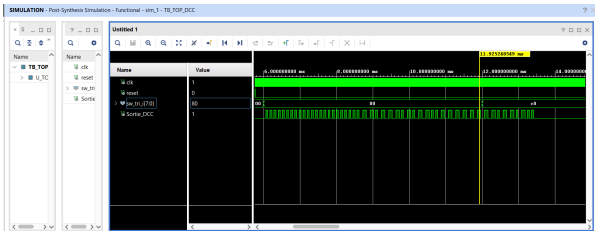
- `Clk` : Horloge 100 MHz (celle de la carte)
- `Reset` : Signal de réinitialisation
- `Fin_Tempo` : Fin de l'attente pour lire la prochaine trame
- `Fin_0` : Fin de l'attente de l'envoi du bit 0, depuis BIT_0
- `Fin_1` : Fin de l'attente de l'envoi du bit 1, depuis BIT_1
- `Reg_DCC` : Bit de poids fort de la trame, depuis Registre_DCC
- `Start_Tempo` : Signal pour demander l'attente avant la lecture de la prochaine trame
- `Go_0` : Signal pour demander l'envoi d'un BIT_0
- `Go_1` : Signal pour demander l'envoi d'un BIT_1
- `Com_Reg` : Signal pour demander soit un Shift, soit un Load

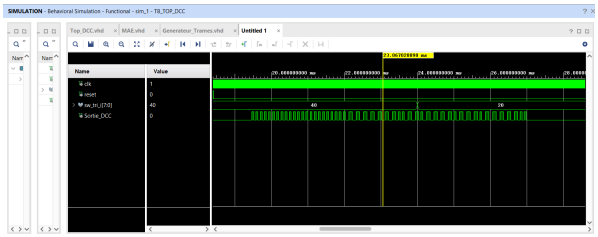
MAE :



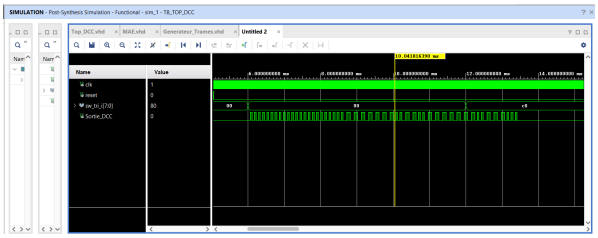
TestBench :

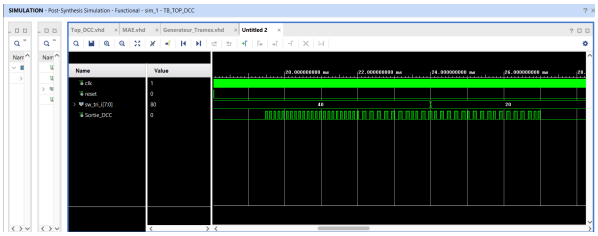
Simulation :





Simulation post-synthèse :





Dans les deux simulations, les deux trames sont identiques.

- Concernant le module de Génération des Trames de Test : compléter le code VHDL en spécifiant les trames que vous voulez envoyer pour tester votre architecture.

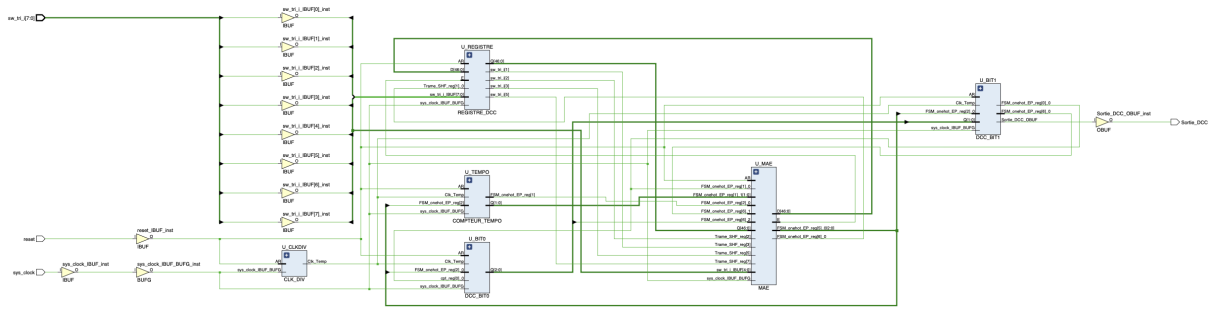
On n'écrit pas toute la trame mais uniquement la commande, car elle se répète. Mais si une commande est de longueur 16, on diminue la taille de son préambule.

Pour les trames, nous avons choisi celles-ci :

- **`switch(7)`** : Vitesse avant maximum avec la commande suivante : `b"01111111"`
- **`switch(6)`** : Vitesse arrière minimum avec la commande suivante : `b"01000010"`
- **`switch(5)`** : Allumage des phares avec la commande suivante : `b"10010000"`
- **`switch(4)`** : Extinction des phares : `b"10000000"`
- **`switch(3)`** : Activation du klaxon : `b"10100100"`
- **`switch(2)`** : Désactivation du klaxon : `b"10100000"`
- **`switch(1)`** : Activation annonceur SNCF : `b"11011110 0 00000001"`
- **`switch(0)`** : Désactivation annonceur SNCF : `b"11011110 0 00000000"`

Ici, c'est le plus grand bit de poids fort qui a la priorité sur la commande choisie. Par exemple, si les interrupteurs 7 et 4 sont tous les deux levés, ce sera la trame de l'interrupteur 7 qui sera générée.

Schéma RTL



Validation

1. Oscilloscope

Lors des tests à l'oscilloscope, nous avons pu observer avec succès l'émission d'une trame.

2. Train

À l'**aide** du **bitstream** généré **et** de la **carte FPGA**, nous avons pu **utiliser** l'**ensemble** des **commandes** que nous avions codées et définies dans le fichier *generateur_trame*.

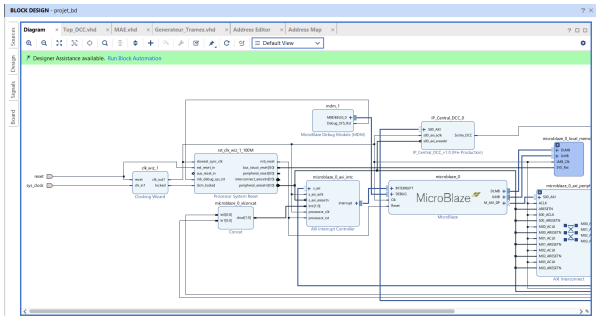
Partie Software

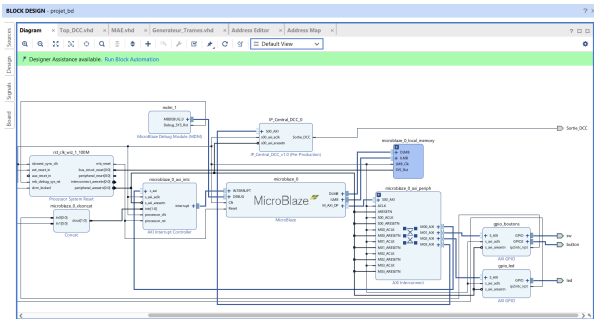
Pour réaliser ce système ***MicroBlaze***, nous avons décidé de :

- Nous baser sur le **générateur de trame** précédent pour faire l'***AXI Wrapper***. On récupère les informations nécessaires envoyées par le *MicroBlaze* dans un **registre** pour générer la trame correspondante.
- Utiliser un **seul registre** : les **8 bits de poids faible** contiennent les valeurs des **sept premiers interrupteurs** de la carte et les **8 bits de poids fort** contiennent l'**adresse du train** à contrôler.
- Utiliser **sept interrupteurs** et **trois boutons** pour la modification et l'envoi des registres :
  - les interrupteurs contrôlent **quelle trame de commande** sera envoyée dans la **sortie *DCC*** (de la même manière que le générateur de trame précédent).
  - les boutons **gauche et central** permettent de choisir l'**adresse de train** à écrire dans le registre à envoyer à l'*IP*.
  - le bouton **droit** permet d'**écrire sur le registre** et donc d'**envoyer son contenu à l'*IP***.
- Utiliser les **LEDs** de la carte pour **afficher l'adresse** du train sélectionnée.

Le **code C** du *MicroBlaze* se contente de fixer les **paramètres généraux** de la trame à envoyer (**adresse du train**, **type de commande**) et envoie ces paramètres à l'*IP* dans un **registre d'interface**. Il faut alors ajouter dans l'*AXI Wrapper* de l'*IP* une **architecture qui décode** les paramètres reçus pour **générer la trame *DCC*** correspondante puis la **transférer au cœur de l'*IP***.

Block Design





Dans ce block design, nous avons configuré deux GPIO (Boutons et LEDs) et créé une *IP*, celle de la ***Centrale DCC***. Nous avons aussi rajouté un port de sortie `Sortie_DCC` connecté à la sortie du même nom de l'*IP Centrale DCC*.

IP Centrale DCC

Pour réaliser l'***AXI Wrapper*** nous nous servons du registre `slv_reg0` :
- bit 24 à 31 : adresse du train
- bit 0 à 7 : interrupteurs

```
reset_interne <= not S_AXI_ARESETN;
TOP: entity work.TOP_DCC
port map
(
    reset      => reset_interne,
    sys_clock  => S_AXI_ACLK,
    sw_tri_i   => slv_reg0(7 downto 0),
    addr_train => slv_reg0(31 downto 24),
    Sortie_DCC => Sortie_DCC
 );
```

(Nous inversons le signal du reset car il est actif à l'état bas)

Nous récupérons les 8 bits de poids fort de `slv_reg0` pour déterminer l'adresse du train et ses 8 bits de poids faible pour choisir quelle trame envoyer dans la MAE.

Nous nous sommes basés sur le générateur de trame pour choisir quelle commande envoyer. Selon le bit de poids fort à 1 de `sw_tri_i`, on envoie la commande :
- **`sw_tri_i(7)`** : Vitesse avant maximum avec la commande suivante : `b"01111111"`
- **`sw_tri_i(6)`** : Vitesse arrière minimum avec la commande suivante : `b"01000010"`
- **`sw_tri_i(5)`** : Allumage des phares avec la commande suivante : `b"10010000"`
- **`sw_tri_i(4)`** : Extinction des phares : `b"10000000"`
- **`sw_tri_i(3)`** : Activation du klaxon : `b"10100100"`
- **`sw_tri_i(2)`** : Désactivation du klaxon : `b"10100000"`
- **`sw_tri_i(1)`** : Activation annonceur SNCF : `b"11011110 0  00000001"`
- **`sw_tri_i(0)`** : Désactivation annonceur SNCF : `b"11011110 0 00000000"`

Comme pour le générateur de trame de la partie précédente, c'est le plus grand bit de poids fort qui a la priorité sur la commande choisie.

GPIOs

GPIO Boutons :
Nous avons rattaché les boutons gauche, droit et central de la carte avec les 8 premiers interrupteurs à ce *GPIO*. C'est un *GPIO dual channel* dont les deux chaînes sont des entrées.

GPIO LED :
Ce *GPIO* est rattaché aux LEDs de la carte. Il ne possède qu'un *channel* de sortie.

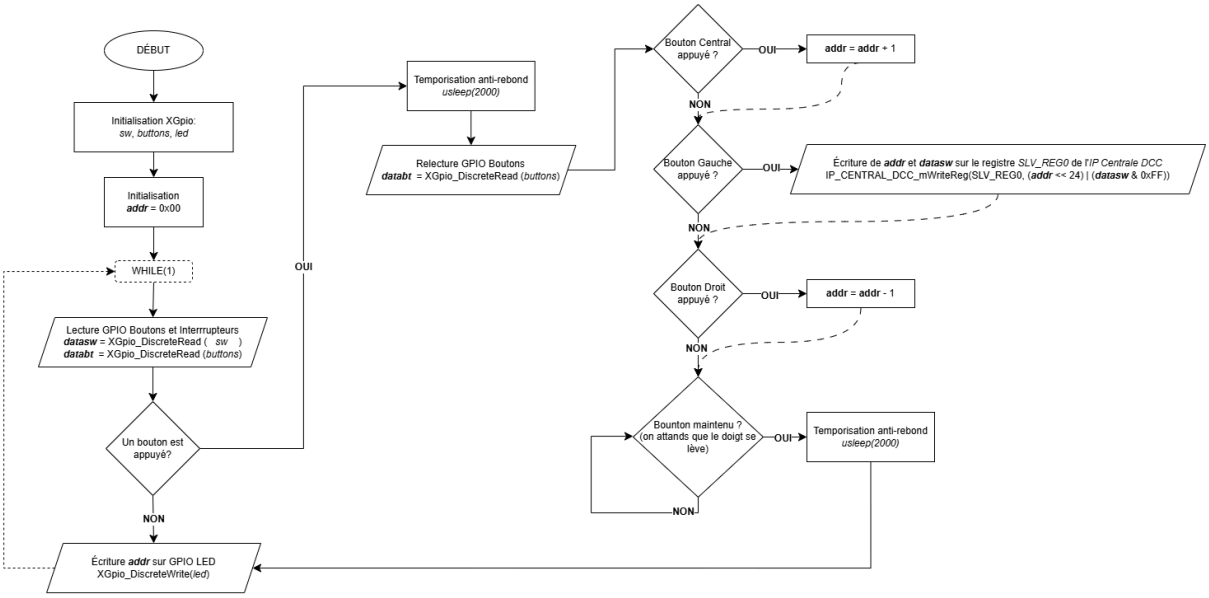
MicroBlaze

Pour **réaliser** le **code C** du ***MicroBlaze***, on **récupère** les **valeurs** en **entrée** du ***GPIO Boutons***.
Selon le bouton appuyé, l'une de ces trois actions se réalise :

- **Bouton central** : incrémentation de la valeur `addr`.
- **Bouton gauche** : décrémentation de la valeur `addr`.
- **Bouton droit** : écriture sur le registre `slv_reg0` de l'*IP Centrale DCC* de la valeur actuelle des interrupteurs et de la valeur `addr`.

Cela nous permet de choisir quel train contrôler pendant l'exécution du programme. De plus, on écrit la valeur courante de `addr` dans le *GPIO LED*, afin d'afficher l'adresse sélectionnée sur les LEDs de la carte.

Organigramme du code C :



Validation

Avec le ***bitstream* généré** et **exporté** dans **Vitis** et le projet application complété, nous avons pu **exécuter** le **code C** du ***MicroBlaze*** sur la carte. À **chaque fois** que l'on **appuie** sur le **bouton d'envoi**, on peut bel et bien voir qu'une **nouvelle commande** est **lancée** dans le train (selon les interrupteurs levés) et il est possible d'envoyer une commande à un train différent en cours d'exécution à partir des deux autres boutons.